

Structure Diffusion Quantization: Learning Stable Coarse-to-Fine Semantic IDs for Generative Recommendation

Anonymous Author(s)

Abstract

Residual Quantization (RQ) has been widely adopted for constructing item semantic IDs in generative recommendation. While previous efforts have focused on incorporating collaborative information through elaborate training objectives, the residual quantization strategy itself has rarely been examined. In particular, RQ still suffers from two fundamental limitations. (I) *Hierarchy*: residual quantization greedily encodes as much information as possible at each stage, yet provides no principled guarantee of the desired coarse-to-fine hierarchy. (II) *Stability*: RQ-based quantizers are known to be sensitive to codebook initialization and checkpoint cherry-picking, resulting in unstable performance and poor reproducibility.

In this paper, we propose Structure Diffusion Quantization (SDQ), which interprets residual quantization stages as discretized graph diffusion at different time steps. SDQ offers three key advantages. First, based on Dirichlet energy, SDQ formally characterizes the common information encoded at each quantization stage, enabling an energy-guided hierarchical organization of semantic IDs. Second, it directly incorporates structural priors into quantization, facilitating collaborative information injection without elaborate loss designs. Third, it improves data utilization by providing broader supervision to code vectors on average, leading to more stable optimization and faster convergence. By enforcing forward-backward consistency in graph diffusion, SDQ-KMeans and SDQ-VAE substantially outperform their RQ-based counterparts in hierarchical structure, training stability, and downstream recommendation performance. Our code is available at <https://anonymous.4open.science/r/SDQ-403>.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Generative recommendation, Vector quantization, Graph diffusion

ACM Reference Format:

Anonymous Author(s). 2018. Structure Diffusion Quantization: Learning Stable Coarse-to-Fine Semantic IDs for Generative Recommendation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Generative recommendation [6, 30] has emerged as a transformative paradigm in the recommendation community due to its natural compatibility with autoregressive and next-token prediction architectures [29], offering the promise of end-to-end modeling, improved scalability, and better explainability. A key component of generative recommendation is the construction of semantic IDs (SIDs) [14, 27], where items are compressed into discrete code sequences through vector quantization of their continuous semantic representations. Consequently, the extent to which SIDs faithfully preserve the intrinsic relationships among items is critical to downstream recommendation performance.

Residual Quantization (RQ) [23] has become the *de facto* choice for item tokenization in generative recommendation, following the pioneering TIGER [30] and OneRec [6, 46] frameworks that adopt RQ-VAE [23] and RQ-KMeans [27], respectively. RQ recursively encodes residual vectors to generate SIDs that preserve rich item information. However, RQ still suffers from critical limitations that may hinder further progress in this area. First, RQ is expected to produce SID sequences in a coarse-to-fine hierarchical manner. Nevertheless, existing RQ-based methods provide no explicit mechanism to guarantee or regulate such behavior. Second, RQ performs item-wise independent encoding, thereby ignoring the item-item relationships inherent in recommendation data. This design limits the ability of SIDs to capture underlying structural priors and results in inefficient and unstable codebook learning. Consequently, RQ-VAE and its variants are highly sensitive to codebook initialization and checkpoint cherry-picking. Follow-up studies typically introduce sophisticated training objectives to impose desirable properties on SIDs [10, 39, 40, 48], such as auxiliary diversity losses for improved and more uniform alignment and contrastive losses for collaborative consistency. Nevertheless, these methods leave the key quantization mechanism unchanged and are often sensitive to carefully tuned hyperparameters.

Item-item graphs, particularly those capturing co-occurrence relationships induced by user interactions [25, 42, 43], provide a principled foundation for defining ‘common’ collaborative information. We therefore introduce Dirichlet energy [7, 47] over item representations, where lower energy indicates that closely related items are represented more similarly in the corresponding vector space. This naturally gives rise to a coarse-to-fine hierarchical quantization criterion: **earlier codes should be encouraged to encode low-energy, common information, while later codes capture higher-energy, fine-grained distinctions.**

Structure Diffusion Quantization (SDQ) proposed in this paper further introduces graph diffusion [1, 4] to model how Dirichlet energy is progressively diffused forward and recovered backward. This enables SDQ to perform quantization in the diffused space at a specific time step, ensuring that the diffused representations carry an appropriate level of common information. The quantized code vectors are then mapped back to the original space through

backward recovery, allowing the already encoded information to be precisely subtracted. Notably, SDQ reduces to vanilla RQ in the absence of structural priors, thereby also maintaining compatibility with existing RQ variants.

Since the exact diffusion process is naturally formulated in an idealized continuous-time setting, we employ numerical discretization for practical implementation [2]. To minimize the mismatch between the forward and backward diffusion processes, we adopt the adaptive DOPRI5 solver [8], which yields lower discretization errors than fixed-step alternatives. However, higher-order solvers introduce additional computational overhead due to repeated intermediate diffusion evaluations, and this cost grows rapidly with the size of the item–item graph. To this end, we employ training subgraph sampling, restricting each propagation step to a subset of nodes sampled directly from the user sequences in the current mini-batch. We develop SDQ-KMeans and SDQ-VAE by integrating SDQ into their respective codebook-learning paradigms. Like RQ-KMeans, SDQ-KMeans is a plug-and-play tokenization method that enables rapid deployment. SDQ-VAE is further trainable and therefore more flexible in incorporating additional SID constraints. We further show, both theoretically and empirically, that SDQ-VAE utilizes data more efficiently than RQ-VAE. This property may stabilize joint codebook optimization, as each code vector receives supervision from a larger number of training samples on average.

In summary, our contributions are as follows:

- We propose SDQ, a principled generalization of RQ that reinterprets residual quantization as a special case of forward–backward graph diffusion. Moreover, the SDQ framework provides theoretical tools for defining and analyzing hierarchical semantic IDs, a necessary step toward controllable item tokenization.
- We analyze the hierarchical and optimization properties of SDQ. Energy dissipation under graph diffusion and the derived gradients reveal improved data utilization and joint codebook optimization, thereby enhancing learning stability.
- We instantiate SDQ as SDQ-KMeans and SDQ-VAE, with numerical discretization and subgraph sampling for practical deployment. Extensive experiments verify its advantages in SID hierarchy, data utilization, optimization stability, and downstream recommendation performance.

2 Preliminaries

Generative Recommendation. Unlike traditional dense-retrieval-based approaches, generative recommendation autoregressively predicts the next token conditioned on the user’s interaction history,

$$\mathbb{P}(v|v_1, v_2, \dots, v_L) = \prod_{k=1}^K \mathbb{P}(s_k^v | s_1^{v_1}, s_2^{v_1}, \dots, s_K^{v_L}, s_1^{v_2}, s_2^{v_2}, \dots, s_{k-1}^{v_L}),$$

where each item $v \in \mathcal{V}$ is first compressed into a semantic ID (SID) sequence $(s_1^v, s_2^v, \dots, s_K^v)$ using vector quantization methods such as RQ-KMeans and RQ-VAE. Specifically, given an item representation $\mathbf{x}_v \in \mathbb{R}^d$, residual quantization starts from $\mathbf{r}_1 = \mathbf{x}_v$ and recursively selects the nearest code vector from each codebook $\mathbf{C}_k \in \mathbb{R}^{M \times d}$:

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \Pi(\mathbf{r}_k; \mathbf{C}_k), \quad \Pi(\mathbf{r}_k; \mathbf{C}_k) := \arg \min_{\mathbf{c}_s \in \mathbf{C}_k} \|\mathbf{r}_k - \mathbf{c}_s\|. \quad (1)$$

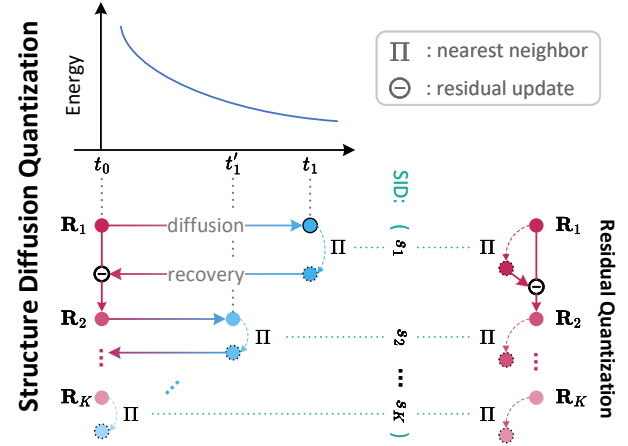


Figure 1: Illustration of how SDQ forms residual cycles from low-energy to high-energy quantization, with RQ interpreted as a special case that operates only at the time point t_0 .

Residual quantization is particularly appealing for generative recommendation because its potential hierarchical structure may facilitate coarse-to-fine next-token prediction. However, such naïve residual framework lacks a formal characterization of hierarchy, making it difficult to analyze or regulate hierarchical behavior in subsequent designs.

Dirichlet energy. Items in recommendation scenarios naturally exhibit non-Euclidean relational structures, such as co-occurrence relationships induced by user interactions. Formally, given an item set $\mathcal{V}$, these relationships can be represented by an adjacency matrix $\mathbf{A} = [w_{ij}] \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, where w_{ij} measures the relational strength between items i and j . For example, two items are expected to have high pairwise similarity if they are co-interacted with by the same user (see Appendix A.3 for construction details). In graph collaborative filtering, let $\mathbf{D} = \text{diag}(d_1, \dots, d_{|\mathcal{V}|})$ denote the diagonal degree matrix of $\mathbf{A}$. The adjacency matrix $\mathbf{A}$ is typically symmetrically normalized as $\tilde{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$, and the corresponding graph Laplacian is defined as $\tilde{\mathbf{L}} = \mathbf{I} - \tilde{\mathbf{A}}$. For an item feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$, its Dirichlet energy over $\tilde{\mathbf{L}}$ is given as

$$\mathcal{E}(\mathbf{X}) := \text{Tr}(\mathbf{X}^\top \tilde{\mathbf{L}} \mathbf{X}) = \frac{1}{2} \sum_{i,j \in \mathcal{V}} w_{ij} \left\| \frac{\mathbf{x}_i}{\sqrt{d_i}} - \frac{\mathbf{x}_j}{\sqrt{d_j}} \right\|_2^2. \quad (2)$$

The Dirichlet energy induced by $\tilde{\mathbf{L}}$ quantifies the local discrepancy between each item representation and those of its neighbors. A lower value of $\mathcal{E}(\mathbf{X})$ indicates that the feature matrix varies more smoothly over the graph. The normalized Dirichlet energy, $\mathcal{E}_{\text{norm}}(\mathbf{X}) = \mathcal{E}(\mathbf{X}) / \|\mathbf{X}\|_F^2$, is also widely used as a scale-invariant measure that eliminates the influence of the overall magnitude.

3 Structure Diffusion Quantization

This section first formulates Structure Diffusion Quantization (SDQ) under continuous graph diffusion equations, establishing a theoretical basis for how node features are diffused (smoothed) forward and recovered (sharpened) backward in residual cycles. It then presents

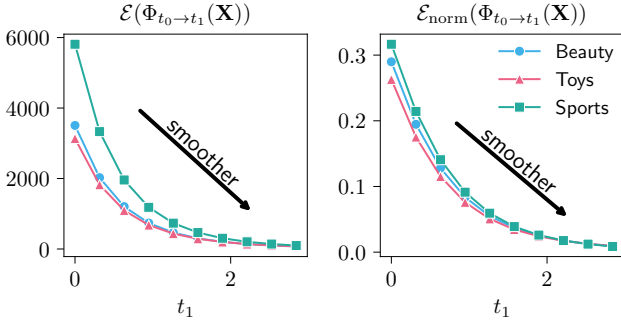


Figure 2: Energy dynamics of $\Phi_{t_0 \rightarrow t_1}(\mathbf{X})$ as t_1 increases. Left: Dirichlet energy. Right: normalized Dirichlet energy.

practical SDQ implementations based on classical discretization schemes, followed by a theoretical analysis of SDQ’s hierarchical property and learning stability.

3.1 SDQ: Continuous Perspective

The naïve residual quantization adopted by RQ-KMeans and RQ-VAE performs greedy encoding independently for each item, and thus lacks a principled guarantee on the amount of Dirichlet energy captured at each stage. Given the adjacency matrix $\tilde{\mathbf{A}}$ that characterizes item relational structures, highly related items are expected to be encoded jointly at the early quantization stages.

Graph diffusion generalizes the heat diffusion equation to non-Euclidean structures. It describes a continuous smoothing process in which the Dirichlet energy is progressively dissipated over the graph until the system reaches an equilibrium state. Compared with layer-by-layer graph convolution [20], the differential-equation perspective further provides a reversible continuous-time formulation for recovering earlier states from later ones; that is

$$\text{Forward: } \mathbf{X}(t_1) = \Phi_{t_0 \rightarrow t_1}(\mathbf{X}(t_0)) := \mathbf{X}(t_0) - \int_{t_0}^{t_1} \tilde{\mathbf{L}}\mathbf{X}(\tau) d\tau, \quad (3)$$

$$\text{Backward: } \mathbf{X}(t_0) = \Phi_{t_1 \rightarrow t_0}(\mathbf{X}(t_1)) := \mathbf{X}(t_1) + \int_{t_0}^{t_1} \tilde{\mathbf{L}}\mathbf{X}(\tau) d\tau. \quad (4)$$

Here, $-\tilde{\mathbf{L}}$ serves as the infinitesimal generator of graph diffusion, smoothing local feature discrepancies over time. Consequently, the Dirichlet energy $\mathcal{E}(\mathbf{X}(t))$ monotonically decreases along the forward diffusion trajectory (as shown in Figure 2). Conversely, the reverse-time equation recovers earlier, less-smoothed states by reversing the diffusion dynamics, thereby sharpening the feature matrix and progressively restoring the Dirichlet energy.

While previous approaches essentially apply vector quantization at the original time $t_0 = 0$, SDQ performs quantization in a diffused space at time t_1 (as illustrated in Figure 1). Specifically, t_1 is set larger at earlier stages to impose stronger smoothing and is gradually reduced to 0 at the final stage. At stage $k = 1, 2, \dots, K$, SDQ first smooths the residual matrix $\mathbf{R}_k$ via $\Phi_{t_0 \rightarrow t_1}$, selects the nearest code vector for each item in the diffused space, and maps the selected code vectors back to the original space through $\Phi_{t_1 \rightarrow t_0}$. The resulting sharpened code vectors can then be subtracted from

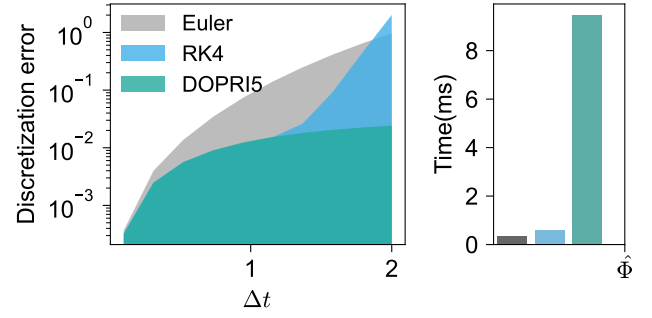


Figure 3: Illustration of discretization errors and computation times across uniformly sampled $\mathbf{A}$ and $\mathbf{X}$.

$\mathbf{R}_k$, producing the residuals for the next quantization stage:

$$\mathbf{R}_{k+1} = \mathbf{R}_k - \Phi_{t_1 \rightarrow t_0} \circ \Pi \circ \Phi_{t_0 \rightarrow t_1}(\mathbf{R}_k), \quad t_0 = 0, \quad t_1 = K - k. \quad (5)$$

The stage-wise setting $t_1 = K - k$ here is an empirical choice motivated by graph convolution used in collaborative filtering. This choice preserves the simplicity and usability of SDQ in this study.

3.2 SDQ: Discretization

The forward smoothing and backward sharpening dynamics induced by Φ are formulated in an idealized continuous-time setting, where the evolution is governed by infinitesimal steps. The discretization schemes introduced below bridge this gap and yield practical implementations of SDQ:

- **Euler discretization.** The Euler scheme provides the simplest first-order approximation to the continuous diffusion dynamics. The forward transition from t to $t + \Delta t$ is approximated by

$$\hat{\Phi}_{t \rightarrow t+\Delta t}(\mathbf{X}(t)) = \mathbf{X}(t) - \Delta t \tilde{\mathbf{L}}\mathbf{X}(t) = (\mathbf{I} - \Delta t \tilde{\mathbf{L}})\mathbf{X}(t).$$

Analogously, the backward transition can be discretized by reversing the integration direction. Euler discretization is known for its computational efficiency, but it may introduce relatively large discretization errors, especially when the step size Δt is not sufficiently small.

- **Fourth-order Runge–Kutta (RK4).** RK4 substantially reduces approximation errors to $O(\Delta t^4)$ by using a weighted combination of four staged increments. Compared with Euler discretization, RK4 provides a higher-order approximation to the continuous trajectory while maintaining a moderate computational cost relative to adaptive methods.
- **Dormand–Prince 5(4) method (DOPRI5).** DOPRI5 is an adaptive Runge–Kutta method that adjusts the step size according to the estimated numerical error between embedded fourth- and fifth-order accurate solutions. It therefore yields a more accurate approximation due to adaptive step sizes, but incurs a non-negligible cost due to additional function evaluations.

We defer the specific forward and backward discretization formulas to Appendix B. As illustrated in Figure 3, we adopt the DOPRI5 scheme to minimize the forward–backward mismatch. Although DOPRI5 incurs additional computational overhead due to its adaptive stepsize control, the overall cost remains manageable owing to the efficiency of SDQ and the modifications introduced below.

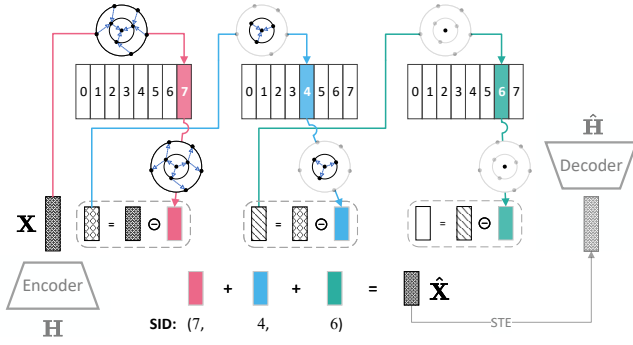


Figure 4: Illustration of SDQ-KMeans (w/o $\square$ modules) and SDQ-VAE. SDQ-KMeans serves as a nearly plug-and-play tokenizer, while SDQ-VAE introduces learnable components for greater flexibility and extensibility.

3.3 Application: SDQ-KMeans & SDQ-VAE

SDQ-KMeans is implemented by combining the classical KMeans algorithm with SDQ. Here, the item feature matrix $X = H \in \mathbb{R}^{|\mathcal{V}| \times p}$ is directly obtained from a pretrained text encoder, *e.g.*, sentence-t5-xl, applied to item text descriptions. At each stage k , KMeans is independently performed on the smoothed residuals $\hat{\Phi}_{t_0 \rightarrow t_1}(R_k)$ to generate the k -th codebook C_k , after which nearest-neighbor search is performed. Since SDQ-KMeans invokes DOPRI5 only $2K$ times, the additional computational cost remains acceptable.

SDQ-VAE is a learnable encoder-SDQ-decoder architecture. As shown in Figure 4, the encoder-SDQ module serves as the tokenizer at the recommendation stage. In this case, the input matrix H is first transformed by an MLP $f(\cdot)$ to obtain the latent representations $X = f(H)$ for quantization. Then, SDQ is applied to X to produce SIDs and the corresponding quantized latent representations for subsequent reconstruction:

$$\hat{H} = g\left(X + \text{sg}(\hat{X}_K - X)\right), \quad \hat{X}_K = \sum_{k'=1}^K \hat{\Phi}_{t_1 \rightarrow t_0} \circ \Pi \circ \hat{\Phi}_{t_0 \rightarrow t_1}(R_{k'}).$$

Here, $\text{sg}(\cdot)$ denotes the stop-gradient operation, which redirects gradients to the encoder through the straight-through estimator. The training objective is defined as

$$\mathcal{L}_{SDQ} = \|\hat{H} - H\|_F^2 + \sum_{k=1}^K \|\hat{X}_k - \text{sg}(X)\|_F^2 + \beta \sum_{k=1}^K \|X - \text{sg}(\hat{X}_k)\|_F^2.$$

Here, β is the commitment weight, and $\beta = 1$ works well empirically. Note that the quantization loss differs slightly from that of RQ-VAE. Instead of independently pulling residuals and selected code vectors together, SDQ-VAE enforces the closeness between X and each partial quantized estimate $\hat{X}_k$. This design empirically improves training stability and convergence speed compared with vanilla RQ-VAE, and is further supported by the theoretical analysis below.

Directly applying DOPRI5 to SDQ-VAE during training is computationally prohibitive, as each function evaluation has a complexity of $O(|A|d)$, where $|A|$ denotes the number of edges in the item graph and d represents the feature dimension. For real-world recommendation systems with massive item sets, full-graph diffusion

becomes infeasible due to the resulting graph scale. To address this challenge, we employ subgraph sampling during training. Since SDQ-VAE learns a tokenizer for downstream *sequential* recommendation, each iteration samples a mini-batch of users and performs SDQ on the induced subgraph constructed from their historical interactions. Notably, SDQ-VAE typically converges within approximately 1% of the training iterations required by RQ-VAE. Therefore, despite the additional cost of DOPRI5, the practical computational overhead remains modest.

3.4 Theoretical Analysis

Energy dissipation under graph diffusion. Under the graph diffusion dynamics $\dot{X}(t) = -\tilde{L}X(t)$, the Dirichlet energy $\mathcal{E}(X(t))$ is non-increasing over time. Since $\tilde{L}$ is symmetric, we have

$$\begin{aligned} \frac{d}{dt} \mathcal{E}(X(t)) &= \frac{d}{dt} \text{Tr}\left(X^\top(t) \tilde{L} X(t)\right) = 2 \text{Tr}\left(X^\top(t) \tilde{L} \dot{X}(t)\right) \\ &= -2 \text{Tr}\left(X^\top(t) \tilde{L}^2 X(t)\right) = -2 \|\tilde{L} X(t)\|_F^2 \leq 0. \end{aligned}$$

Therefore, graph diffusion monotonically dissipates Dirichlet energy (as shown in Figure 2) while progressively smoothing the feature matrix over the item-item graph. By applying longer diffusion times at earlier quantization stages, SDQ is theoretically justified to exhibit coarse-to-fine hierarchical behavior.

Connection to residual quantization. When the normalized adjacency matrix $\tilde{A}$ reduces to the identity matrix I , no structural prior is incorporated and the corresponding graph Laplacian becomes $\tilde{L} = I - \tilde{A} = 0$. Consequently, both $\Phi_{t_0 \rightarrow t_1}$ and $\Phi_{t_1 \rightarrow t_0}$ reduce to the identity mapping. In this case, SDQ yields

$$R_{k+1} = R_k - \Pi(R_k),$$

which is exactly the matrix formulation of residual quantization. Therefore, RQ can be viewed as a special case of SDQ in the absence of structural priors.

Data efficiency and stability. Fix the nearest-neighbor assignments and solver steps. Let $P_k \in \{0, 1\}^{|\mathcal{V}| \times M}$ denote the one-hot assignment matrix and B_k the backward recovery matrix, so that the recovered code vectors are $B_k P_k C_k$. The forms of B_k and the proofs are given in Appendices B and C, respectively.

PROPOSITION 3.1 (CODEBOOK GRADIENT OF SDQ-VAE). *At a given training step, the gradient of $\mathcal{L}_{SDQ}$ w.r.t. the k -th codebook is*

$$\begin{aligned} \nabla_{C_k} \mathcal{L}_{SDQ} &= 2 \sum_{j=1}^K (K - \max\{k, j\} + 1) P_k^\top B_k^\top B_j P_j C_j \\ &\quad - 2(K - k + 1) P_k^\top B_k^\top X. \end{aligned} \quad (6)$$

The first term in Eq. (6) accounts for the intra- and inter-codebook dependencies during training. In particular, the matrix $P_k^\top B_k^\top B_k P_k$ characterizes the interactions among vectors within the same codebook. Because the backward recovery matrix B_k integrates multi-hop item-item structures, it is typically not highly sparse. This property facilitates joint codebook learning.

The second term in Eq. (6) isolates the direct dependence on the item representations X and motivates the following definition.

Table 1: Overall performance comparison. Results (%) are averaged across five independent runs and reported in the format of mean_(std). Standard deviations are highlighted in red when the relative standard deviation, std/mean, exceeds 0.05.

Beauty					Toys				Sports				
	HR@5	HR@10	NDCG@5	NDCG@10	HR@5	HR@10	NDCG@5	NDCG@10	HR@5	HR@10	NDCG@5	NDCG@10	
Conventional recommendation methods (stability: high overall)													
MF [31]	2.70 _(0.04)	4.47 _(0.04)	1.70 _(0.02)	2.27 _(0.03)	2.60 _(0.07)	4.26 _(0.09)	1.72 _(0.03)	2.25 _(0.03)	1.77 _(0.02)	2.98 _(0.07)	1.13 _(0.02)	1.52 _(0.03)	
LightGCN [12]	2.89 _(0.05)	4.80 _(0.09)	1.80 _(0.03)	2.42 _(0.04)	2.95 _(0.04)	4.69 _(0.06)	1.92 _(0.02)	2.47 _(0.02)	1.93 _(0.03)	3.18 _(0.06)	1.24 _(0.02)	1.64 _(0.02)	
Caser [35]	2.62 _(0.12)	4.27 _(0.13)	1.68 _(0.08)	2.21 _(0.08)	1.82 _(0.14)	2.86 _(0.16)	1.20 _(0.07)	1.54 _(0.08)	1.36 _(0.10)	2.25 _(0.10)	0.85 _(0.08)	1.13 _(0.07)	
GRU4Rec [13]	2.43 _(0.11)	4.05 _(0.15)	1.54 _(0.06)	2.06 _(0.08)	2.67 _(0.11)	4.09 _(0.13)	1.81 _(0.08)	2.27 _(0.08)	1.19 _(0.07)	1.97 _(0.07)	0.76 _(0.07)	1.02 _(0.07)	
SASRec [19]	4.26 _(0.17)	6.63 _(0.21)	2.75 _(0.12)	3.51 _(0.13)	5.08 _(0.11)	7.48 _(0.18)	3.45 _(0.08)	4.22 _(0.09)	2.25 _(0.05)	3.57 _(0.07)	1.47 _(0.03)	1.89 _(0.02)	
BERT4Rec [33]	4.89 _(0.10)	7.17 _(0.16)	3.31 _(0.08)	4.05 _(0.09)	4.65 _(0.13)	6.73 _(0.25)	3.24 _(0.10)	3.91 _(0.13)	2.50 _(0.09)	3.94 _(0.09)	1.65 _(0.06)	2.11 _(0.07)	
Constraint-guided generative recommendation methods (stability: moderate overall)													
ETEGRec [26]	3.88 _(0.11)	6.01 _(0.21)	2.57 _(0.09)	3.25 _(0.12)	3.38 _(0.14)	5.07 _(0.27)	2.29 _(0.11)	2.84 _(0.14)	2.57 _(0.13)	3.95 _(0.19)	1.70 _(0.08)	2.14 _(0.10)	
LETTER [40]	4.39 _(0.16)	6.74 _(0.13)	2.86 _(0.11)	3.61 _(0.08)	3.88 _(0.12)	6.09 _(0.21)	2.57 _(0.06)	3.28 _(0.08)	2.93 _(0.13)	4.62 _(0.32)	1.88 _(0.09)	2.43 _(0.15)	
DIGER [10]	4.37 _(0.36)	6.64 _(0.62)	2.92 _(0.24)	3.65 _(0.32)	3.88 _(0.29)	5.87 _(0.47)	2.62 _(0.19)	3.26 _(0.24)	2.49 _(0.14)	3.78 _(0.16)	1.65 _(0.11)	2.07 _(0.11)	
R3-VAE [39]	4.72 _(0.24)	7.10 _(0.32)	3.13 _(0.16)	3.89 _(0.19)	4.47 _(0.17)	6.61 _(0.22)	3.06 _(0.13)	3.74 _(0.13)	2.76 _(0.12)	4.30 _(0.16)	1.80 _(0.08)	2.29 _(0.09)	
TIGER-framework quantization variants (stability: poor overall)													
RQ-VAE	PQ-KMeans [17]	4.48 _(0.15)	6.88 _(0.12)	2.99 _(0.07)	3.76 _(0.06)	4.36 _(0.21)	6.68 _(0.24)	2.90 _(0.14)	3.65 _(0.15)	2.58 _(0.05)	4.01 _(0.12)	1.69 _(0.04)	2.15 _(0.06)
	RQ-KMeans [27]	4.48 _(0.27)	6.83 _(0.38)	2.98 _(0.21)	3.74 _(0.25)	3.94 _(0.36)	6.09 _(0.43)	2.65 _(0.27)	3.35 _(0.29)	1.82 _(0.13)	3.01 _(0.20)	1.17 _(0.08)	1.56 _(0.10)
	PQ-VAE [38]	4.51 _(0.18)	6.84 _(0.17)	3.02 _(0.14)	3.77 _(0.14)	4.23 _(0.12)	6.39 _(0.21)	2.85 _(0.11)	3.54 _(0.12)	1.76 _(0.13)	2.85 _(0.21)	1.13 _(0.09)	1.48 _(0.11)
	STE [30]	4.42 _(0.07)	6.68 _(0.21)	2.98 _(0.05)	3.71 _(0.09)	4.05 _(0.26)	6.11 _(0.31)	2.74 _(0.21)	3.40 _(0.21)	2.26 _(0.34)	3.57 _(0.53)	1.47 _(0.24)	1.89 _(0.30)
	Gumbel [16]	2.21 _(0.40)	3.40 _(0.50)	1.46 _(0.30)	1.85 _(0.33)	0.62 _(0.13)	0.98 _(0.12)	0.41 _(0.10)	0.52 _(0.10)	0.78 _(0.54)	1.36 _(0.84)	0.52 _(0.33)	0.71 _(0.42)
	Rotation [9]	4.22 _(0.18)	6.33 _(0.24)	2.83 _(0.15)	3.51 _(0.17)	3.78 _(0.17)	5.75 _(0.21)	2.58 _(0.12)	3.22 _(0.13)	2.41 _(0.07)	3.79 _(0.05)	1.57 _(0.06)	2.01 _(0.06)
SimVQ [49]	4.45 _(0.07)	6.79 _(0.08)	2.98 _(0.07)	3.73 _(0.07)	4.15 _(0.13)	6.26 _(0.06)	2.83 _(0.08)	3.50 _(0.04)	2.37 _(0.49)	3.75 _(0.71)	1.54 _(0.36)	1.99 _(0.42)	
SDQ-KMeans		5.01 _(0.08)	7.61 _(0.13)	3.39 _(0.07)	4.22 _(0.08)	4.47 _(0.20)	6.83 _(0.24)	3.05 _(0.16)	3.80 _(0.17)	2.75 _(0.07)	4.35 _(0.17)	1.79 _(0.06)	2.31 _(0.09)
Gain vs. Best KMeans		+11.9%	+10.7%	+13.5%	+12.4%	+2.4%	+2.2%	+4.9%	+4.2%	+6.6%	+8.4%	+6.0%	+7.2%
SDQ-VAE		4.99 _(0.17)	7.54 _(0.26)	3.37 _(0.14)	4.19 _(0.15)	4.73 _(0.07)	7.18 _(0.16)	3.19 _(0.07)	3.98 _(0.07)	3.03 _(0.12)	4.58 _(0.12)	2.00 _(0.08)	2.50 _(0.08)
Gain vs. Best VAE		+10.5%	+10.2%	+11.5%	+11.1%	+11.8%	+12.5%	+12.0%	+12.3%	+25.8%	+21.0%	+27.8%	+24.1%

Definition 3.2 (Codebook data utilization ratio). For codebook $C_k \in \mathbb{R}^{M \times d}$, its data utilization ratio is defined as

$$\text{DUR}(C_k) := \frac{\|\mathbf{P}_k^\top \mathbf{B}_k^\top\|_0}{M|\mathcal{V}|}, \quad k = 1, 2, \dots, K. \quad (7)$$

Since $\|\cdot\|_0$ counts nonzero entries, $\text{DUR}(C_k)$ is the fraction of code-item pairs with nonzero supervision coefficients.

COROLLARY 3.3 (CODEBOOK GRADIENT OF RQ). *When no structural prior is used, $\tilde{\mathbf{L}} = \mathbf{0}$ and $\mathbf{B}_j = \mathbf{I}$ at every stage j . Proposition 3.1 then reduces to*

$$\nabla_{C_k} \mathcal{L}_{\text{SDQ}} = 2 \sum_{j=1}^K (K - \max\{k, j\} + 1) \mathbf{P}_k^\top \mathbf{P}_j \mathbf{C}_j - 2(K - k + 1) \mathbf{P}_k^\top \mathbf{X},$$

where $\mathbf{P}_k^\top \mathbf{P}_k$ is diagonal and $\text{DUR}(C_k) = \frac{\|\mathbf{P}_k^\top\|_0}{M|\mathcal{V}|} \equiv \frac{1}{M}$.

The two coefficient matrices highlight the fundamental distinction between SDQ and RQ. For codebook learning, the RQ coefficient matrix, $\mathbf{P}_k^\top \mathbf{P}_k$, is diagonal because the assignment columns have disjoint supports. In contrast, backward diffusion in SDQ generally causes these columns to overlap after multiplication by $\mathbf{B}_k$, making $\mathbf{P}_k^\top \mathbf{B}_k^\top \mathbf{B}_k \mathbf{P}_k$ non-diagonal. Consequently, each nonzero off-diagonal entry introduces coupling between the optimization of the corresponding code vectors. Regarding the data utilization ratio, RQ exploits only directly assigned code vector-item pairs, whereas the additional factor $\mathbf{B}_k^\top$ in SDQ enables structurally related items to

contribute nonzero coefficients in (7). Accordingly, SDQ has a data utilization ratio no lower than that of RQ whenever backward recovery retains the directly assigned coefficients; it is strictly higher whenever structurally related items introduce additional nonzero coefficients in $\mathbf{P}_k^\top \mathbf{B}_k^\top$. Moreover, when this additional supervision is reasonably balanced across code vectors, it is expected to reduce the variance of sample-level gradient estimates. This offers one possible explanation for the improved training stability and convergence observed in SDQ-VAE.

4 Experiments

We conduct extensive experiments to investigate the following research questions: **RQ1**: How does SDQ compare with existing quantization strategies in downstream generative recommendation? **RQ2**: Does SDQ effectively induce the desired coarse-to-fine SID hierarchy while improving data utilization? **RQ3**: How do diffusion discretization and quantization loss influence the performance of SDQ?

4.1 Experimental Setup

Datasets. We evaluate on Beauty, Toys, and Sports, three widely adopted public datasets [19, 30] from the Amazon Reviews. For each dataset, we apply a 5-core filtering strategy, retaining only users and items with at least five interactions. Following the leave-one-out evaluation protocol, we use each user's most recent interaction

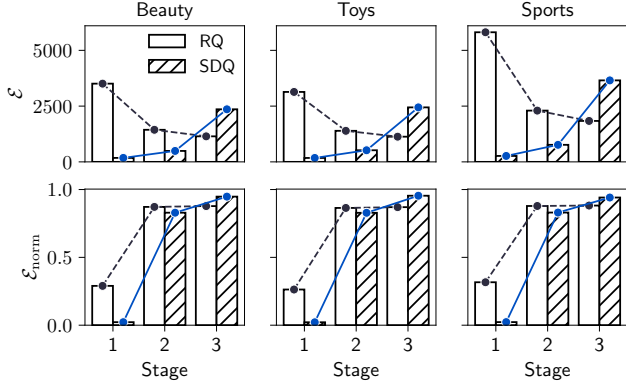


Figure 5: Hierarchy. SDQ exhibits a clear low-to-high energy hierarchy, whereas RQ does not.

for testing and the second most recent interaction for validation. To construct semantic item representations, we encode the title, category, and brand fields using sentence-t5-xl [30]. Detailed dataset statistics are provided in Table 4.

Evaluation Metrics. We evaluate recommendation performance using two widely adopted ranking metrics: HR@N (Hit Rate) and NDCG@N (Normalized Discounted Cumulative Gain). HR@N measures whether the ground-truth item appears among the top-N recommendations, whereas NDCG@N additionally accounts for its ranking position by assigning larger gains to correctly predicted items ranked closer to the top. For generative recommenders, we employ constrained beam search with a beam size of 20 to generate candidate semantic IDs.

Baselines. We organize the baselines into three groups. The first comprises conventional recommenders: MF [31], LightGCN [12], Caser [35], GRU4Rec [13], SASRec [19], and BERT4Rec [33], covering matrix factorization, GNN, CNN, RNN, and Transformer architectures. The second consists of generative recommendation methods that improve semantic IDs through elaborate training objectives or auxiliary constraints: ETEGRec [26], LETTER [40], DIGER [10], and R3-VAE [39]. The third directly modifies the quantization strategy or its discrete optimization: PQ-KMeans [17], RQ-KMeans [27], PQ-VAE [38], and RQ-VAE variants based on STE [30], Gumbel [16], Rotation [9], and SimVQ [49].

Implementation Details. For each baseline, we independently tune the hyperparameters on each dataset to obtain optimal performance. Because SDQ is designed to improve training stability and robustness across datasets, we use a largely shared hyperparameter configuration for SDQ whenever possible; the complete settings are provided in Appendix A. For both RQ-VAE and SDQ-VAE, we apply the commonly used Sinkhorn regularization [45] at the final quantization stage. For downstream recommendation, all generative recommenders employ the same T5 architecture with six layers and four attention heads. We report the mean and standard deviation over five independent runs with random seeds 0, 1, 2, 3, 4.

4.2 Overall Recommendation Performance

Table 1 compares SDQ with conventional recommenders, constraint-guided generative methods, and TIGER-framework quantization

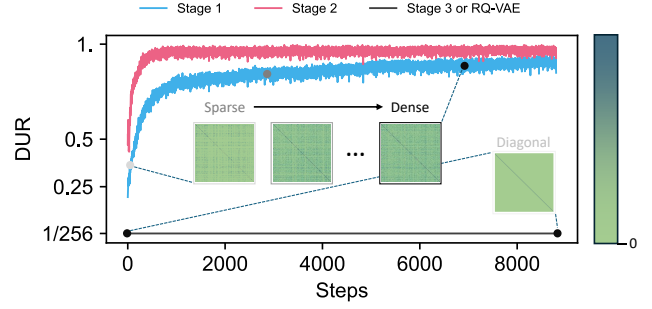


Figure 6: Data utilization. SDQ achieves a substantially higher Data Utilization Ratio (DUR), accompanied by a denser $P^T B^T B P$ (green squares), whereas RQ exhibits low DUR.

variants. We analyze the results from the complementary perspectives of stability and recommendation accuracy.

Stability. The results reveal a pronounced stability gap between conventional and generative recommendation methods. For conventional recommenders, run-to-run variation primarily arises from stochastic model optimization. Generative recommenders, however, must additionally learn semantic IDs before downstream training; randomness in codebook optimization and discrete code assignment therefore introduces a second source of variation. The third block isolates this quantization-level instability more directly. RQ-KMeans exceeds the relative-standard-deviation threshold in all 12 result cells. Rotation Trick and SimVQ partially alleviate the associated optimization difficulty by modifying or replacing the straight-through estimator used for discrete assignments, improving stability at the cost of recommendation accuracy in some settings. Constraint-guided methods provide a more indirect remedy by encouraging semantic IDs to satisfy predefined properties. Although such objectives may reduce fluctuations in the learned representations, they do not fundamentally alter the assignment and update mechanisms that give rise to quantization uncertainty. Their repeatedly highlighted deviations and additional hyperparameter requirements reflect this limitation. By contrast, SDQ redesigns the quantization mechanism. Its broader supervision over code vectors and coupled update scheme improve the quality of gradient estimation during codebook learning. Consequently, SDQ-VAE produces no highlighted deviations and achieves stability comparable to that of conventional baselines, in agreement with our theoretical analysis.

Accuracy. Within the TIGER framework, SDQ-KMeans outperforms both PQ-KMeans and RQ-KMeans across all 12 dataset-metric combinations, yielding relative gains of 2.2%–13.5% over the strongest KMeans counterpart in each setting. SDQ-VAE likewise surpasses PQ-VAE and every RQ-VAE variant in all 12 settings, with larger relative gains of 10.2%–27.8% over the strongest VAE counterpart. These consistent improvements under both KMeans- and VAE-based implementations indicate that replacing independent residual quantization with graph diffusion is effective across different underlying quantizers. Despite not relying on elaborate constraint-based objectives, SDQ-VAE achieves higher mean performance than ETEGRec, LETTER, DIGER, R3-VAE, and the RQ-VAE

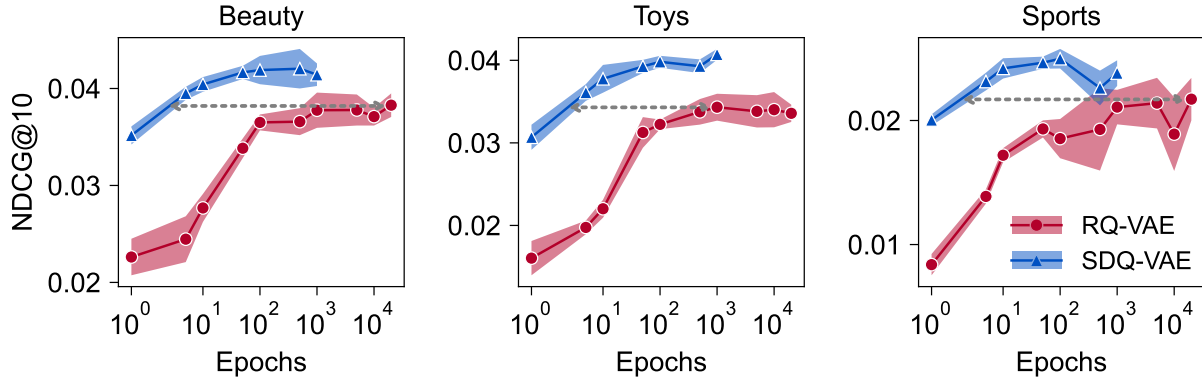


Figure 7: Convergence. Compared with SDQ-VAE, RQ-VAE requires more training epochs to converge and exhibits significantly higher optimization variance. The wall-clock training times are reported in Table 7.

variants in 11 of the 12 settings. The only exception is HR@10 on Sports, where LETTER performs marginally better. This result demonstrates the effectiveness of incorporating collaborative information directly into the quantization process through graph diffusion. It is worth noting that SASRec outperforms all generative methods on Toys, but the two represent fundamentally different recommendation paradigms and are not intended as the primary basis for judging the effectiveness of generative recommendation. For consistency with prior work, we employ a standard T5 architecture, whereas a more capable recommender may further improve recommendation accuracy and potentially surpass SASRec [40].

4.3 Empirical Analysis

Hierarchy. Figure 5 shows that SDQ exhibits the intended coarse-to-fine hierarchical behavior: low-energy components associated with common interaction patterns are quantized first, followed by high-energy components that capture fine-grained distinctions. This ordering follows from the longer forward diffusion applied at earlier quantization stages, which progressively suppresses high-energy graph components before residual quantization. In contrast, RQ performs all residual assignments at the same time point t_0 and consequently provides no mechanism for organizing information according to (normalized) Dirichlet energy.

Data utilization. Figure 6 shows that SDQ achieves a substantially higher codebook Data Utilization Ratio (DUR) than RQ. By Definition 3.2, the higher DUR means that a larger proportion of code-item pairs receive nonzero supervision through $\mathbf{P}_k^T \mathbf{B}_k^T$. The gradual increase of DUR in both Stage 1 and Stage 2 is consistent with the initially non-uniform code assignments: early in training, supervision is concentrated on a limited subset of code vectors and becomes more broadly distributed as optimization proceeds. In addition, the denser $\mathbf{P}^T \mathbf{B}^T \mathbf{B} \mathbf{P}$ reveals non-diagonal dependencies among code vectors, unlike the diagonal structure induced by RQ assignments. These dependencies enable joint learning within and across codebooks. Thus, backward diffusion broadens the effective supervision of individual code vectors while coupling their updates through structurally related items.

Convergence. Figure 7 shows that SDQ-VAE converges within a substantially shorter training horizon, whereas RQ-VAE requires

Table 2: Effect of diffusion discretization on recommendation performance. DOPRI5 achieves the best HR@10 and NDCG@10 in all settings.

SDQ Solver		Beauty		Toys		Sports	
		HR@10	NDCG@10	HR@10	NDCG@10	HR@10	NDCG@10
KMeans	Euler	5.87 _(0.04)	3.20 _(0.04)	5.01 _(0.11)	2.77 _(0.11)	3.44 _(0.11)	1.80 _(0.08)
	RK4	6.74 _(0.17)	3.70 _(0.08)	6.27 _(0.09)	3.50 _(0.05)	3.75 _(0.38)	1.98 _(0.22)
	DOPRI5	7.61 _(0.13)	4.22 _(0.08)	6.83 _(0.24)	3.80 _(0.17)	4.35 _(0.17)	2.31 _(0.09)
VAE	Euler	6.64 _(0.28)	3.75 _(0.17)	5.94 _(0.18)	3.30 _(0.14)	3.94 _(0.09)	2.12 _(0.06)
	RK4	7.54 _(0.06)	4.18 _(0.02)	6.90 _(0.15)	3.84 _(0.09)	4.44 _(0.09)	2.38 _(0.05)
	DOPRI5	7.54 _(0.26)	4.19 _(0.15)	7.18 _(0.16)	3.98 _(0.07)	4.58 _(0.12)	2.50 _(0.08)

more epochs and exhibits considerably greater optimization variance. This observation highlights the optimization challenges inherent in RQ-based SID learning. It also supports the qualitative explanation presented in Section 4.2: broader supervision and coupled codebook updates may reduce the variance of sample-level gradient estimates, thereby stabilizing codebook optimization. Unfortunately, the gains in convergence efficiency are partially offset by the computationally more expensive forward-backward DOPRI5 discretization, which incurs additional overhead at each training iteration. Nevertheless, as reported in Table 7, SDQ-VAE still achieves competitive overall training efficiency and retains substantial potential for further acceleration.

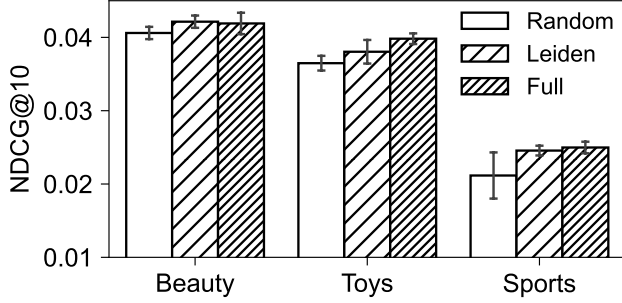
4.4 Ablation Study

Discretization. Table 2 shows that DOPRI5 achieves the best HR@10 and NDCG@10 across all 12 settings. For SDQ-KMeans, performance consistently improves from Euler to RK4 and further to DOPRI5, while SDQ-VAE also benefits from the higher-order solver. These results demonstrate that reducing the numerical discrepancy between forward diffusion and backward recovery is important for SDQ. The adaptive error control of DOPRI5 therefore justifies its additional computational cost, as reported in Table 7.

Quantization loss. We further ablate in Table 3 the quantization objective while keeping the SDQ architecture unchanged. Keeping $\mathcal{L}_{\text{RQ}}$ fixed (see Appendix D.2 for definition), replacing RQ-VAE

Table 3: Effect of the quantization objective on downstream recommendation performance.

VAE Loss	Beauty		Toys		Sports	
	HR@10	NDCG@10	HR@10	NDCG@10	HR@10	NDCG@10
RQ $\mathcal{L}_{RQ}$	6.68 _(0.21)	3.71 _(0.09)	6.11 _(0.31)	3.40 _(0.21)	3.57 _(0.53)	1.89 _(0.30)
SDQ $\mathcal{L}_{RQ}$	7.30 _(0.31)	4.05 _(0.15)	6.90 _(0.14)	3.88 _(0.07)	4.26 _(0.40)	2.31 _(0.25)
SDQ $\mathcal{L}_{SDQ}$	7.54 _(0.26)	4.19 _(0.15)	7.18 _(0.16)	3.98 _(0.07)	4.58 _(0.12)	2.50 _(0.08)

**Figure 8: Effect of inference subgraph sampling.**

with SDQ-VAE improves NDCG@10 by 9.2%, 14.0%, and 22.3% on Beauty, Toys, and Sports, respectively, demonstrating the benefit of diffusion-based quantization itself. Replacing $\mathcal{L}_{RQ}$ with $\mathcal{L}_{SDQ}$ then yields a further 3.4%, 2.7%, and 8.2% improvement in NDCG@10. These consistent improvements show that the SDQ objective provides a complementary advantage beyond diffusion.

Inference subgraph sampling. Figure 8 examines whether subgraph sampling should also be applied at inference for consistency. Random subgraph sampling noticeably degrades recommendation performance, suggesting that arbitrary node subsets discard structural priors essential for graph diffusion. In contrast, Leiden community sampling [37] performs substantially better by preserving densely connected local structures and retaining more of the graph information used by SDQ. Since SID generation requires only a limited number of propagation steps, full-graph inference remains the most reliable default in our experiments. For large-scale settings where full-graph propagation is costly, Leiden sampling provides a practical alternative that reduces inference overhead while maintaining structural coherence.

5 Related Work

Generative recommendation formulates item prediction as autoregressive generation [34, 44], providing next-token prediction as a unified interface for retrieval and ranking. P5 [11] casts recommendation into the natural-language space, whereas TIGER [30] represents items with semantic IDs drawn from a dedicated discrete vocabulary. This reformulation has since driven generative recommendation toward unified, end-to-end architectures. The OneRec series [6, 46] exemplifies this trend by integrating retrieval and ranking in industrial recommendation. Unlike natural language, however, where subword units are grounded in textual corpora [21], recommendation has no canonical way to tokenize items [14]. Tokenizer quality therefore becomes a key determinant of downstream

performance, making vector quantization central to learning effective item tokenizers.

Vector quantization enables the compression of continuous representations containing mixed item semantics into discrete code vectors. VQ-VAE [38] and RQ-VAE [23] facilitate discrete representation learning within encoder-decoder architectures by employing the straight-through estimator to circumvent the non-differentiability of nearest-neighbor assignments. However, codebook optimization remains challenging due to gradient approximation errors, sparse codebook updates [15], and sensitivity to initialization and hyperparameter choices [22]. To address these issues, recent approaches improve gradient propagation via the rotation trick [9], couple code-vector updates through a shared transformation in SimVQ [49], or eliminate codebook learning altogether with FSQ [28]. Beyond stable codebook optimization, recommendation tokenization must also preserve collaborative relationships among items [24]. Subsequent studies refine SIDs through semantic and collaborative regularization in LETTER [40], differentiable tokenization in DIGER [10], and semantic cohesion and preference discrimination regularization in R3-VAE [39]. Overall, existing approaches continue to rely heavily on carefully designed training objectives to enforce desirable properties of SIDs, while overlooking how the quantization strategy itself affects hierarchical structure and learning stability.

Structural information, particularly that derived from interaction data [42], has been extensively studied in recommendation. In collaborative filtering [12, 32, 41], graph-based propagation has proven particularly effective for smoothing user and item representations. Beyond discrete graph convolutions, graph diffusion [1, 4] offers a continuous-time perspective on message propagation. GRAND [1] and GRAND++ [36] formulate GNNs as discretizations of underlying diffusion PDEs, thereby enabling continuous-depth propagation while alleviating the oversmoothing induced by repeated graph convolutions. This perspective has also been introduced into recommendation: LT-OCF [5] learns the integration time for ODE-based collaborative filtering, whereas BSPM [3] directly processes the interaction matrix through continuous-time blurring and sharpening. However, these methods operate on continuous embeddings or prediction scores rather than on the quantization process itself. In contrast, SDQ incorporates forward-backward graph diffusion into residual quantization, enabling structural information to directly govern SID hierarchy formation and codebook learning.

6 Conclusion

This work revisits residual quantization for generative recommendation from a graph-diffusion perspective. We propose Structure Diffusion Quantization (SDQ), a principled extension of RQ that applies a forward-backward diffusion cycle at each quantization stage. Using Dirichlet energy to characterize shared information, SDQ defines a coarse-to-fine SID hierarchy and incorporates structural priors into quantization. Our analysis shows that backward recovery expands codebook supervision and couples code-vector updates, improving optimization stability. SDQ-KMeans and SDQ-VAE consistently outperform RQ-based alternatives in hierarchy quality, data utilization, training stability, and recommendation performance.

References

- [1] Ben Chamberlain, James Rowbottom, Maria I Gorinova, Michael Bronstein, Stefan Webb, and Emanuele Rossi. 2021. Grand: Graph neural diffusion. In *International Conference on Machine Learning*. PMLR, 1407–1418.
- [2] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. 2018. Neural ordinary differential equations. *Advances in Neural Information Processing Systems (NeurIPS)* 31 (2018).
- [3] Jeongwhan Choi, Seoyoung Hong, Noseong Park, and Sung-Bae Cho. 2023. Blurring-sharpening process models for collaborative filtering. In *International ACM Conference on Research and Development in Information Retrieval (SIGIR)*. 1096–1106.
- [4] Jeongwhan Choi, Seoyoung Hong, Noseong Park, and Sung-Bae Cho. 2023. Gread: Graph neural reaction-diffusion networks. In *International Conference on Machine Learning (ICML)*. PMLR, 5722–5747.
- [5] Jeongwhan Choi, Jinsung Jeon, and Noseong Park. 2021. LT-OCF: Learnable-time ode-based collaborative filtering. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 251–260.
- [6] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).
- [7] Francesco Di Giovanni, James Rowbottom, Benjamin P Chamberlain, Thomas Markovich, and Michael M Bronstein. 2023. Understanding convolution on graphs via energies. *Transactions on Machine Learning Research (TMLR)* (2023).
- [8] John R Dormand and Peter J Prince. 1980. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics (JCAM)* 6, 1 (1980), 19–26.
- [9] Christopher Fifty, Ronald Junkins, Dennis Duan, Aniketh Iyengar, Jerry Liu, Ehsan Amid, Sebastian Thrun, and Christopher Ré. 2025. Restructuring vector quantization with the rotation trick. In *International Conference on Learning Representations (ICLR)*, Vol. 2025. 19153–19188.
- [10] Junchen Fu, Xuri Ge, Alexandros Karatzoglou, Ioannis Arapakis, Suzan Verberne, Joemon M Jose, and Zhaochun Ren. 2026. Differentiable semantic id for generative recommendation. *arXiv preprint arXiv:2601.19711* (2026).
- [11] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *ACM Conference on Recommender Systems (RecSys)*. 299–315.
- [12] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *International ACM Conference on Research and Development in Information Retrieval (SIGIR)*. 639–648.
- [13] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*.
- [14] Wenye Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. 2023. How to index item ids for recommendation foundation models. In *International ACM Conference on Research and Development in Information Retrieval (SIGIR)*. 195–204.
- [15] Minyoung Huh, Brian Cheung, Pulkit Agrawal, and Phillip Isola. 2023. Straightening out the straight-through estimator: Overcoming optimization challenges in vector quantized networks. In *International Conference on Machine Learning (ICML)*. PMLR, 14096–14113.
- [16] Eric Jang, Shixiang Gu, and Ben Poole. 2017. Categorical Reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations (ICLR)*.
- [17] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 33, 1 (2010), 117–128.
- [18] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative recommendation with semantic IDs: A practitioner’s handbook. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 6420–6425.
- [19] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *IEEE International Conference on Data Mining (ICDM)*. IEEE, 197–206.
- [20] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.
- [21] Taku Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. 66–75.
- [22] Adrian Łańcucki, Jan Chorowski, Guillaume Sanchez, Ricard Marxer, Nanxin Chen, Hans JGA Dolfing, Sameer Khurana, Tanel Alumäe, and Antoine Laurent. 2020. Robust training of vector quantized bottleneck models. In *International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–7.
- [23] Doyup Lee, Chihyeon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive image generation using residual quantization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11523–11532.
- [24] Yu Liang, Zhongjin Zhang, Yuxuan Zhu, Kerui Zhang, Zhiluo Han Guo, Wenhao Zhou, Zonqi Yang, Kangle Wu, Yabo Ni, Anxiang Zeng, et al. 2026. Rethinking generative recommender tokenizer: Recsys-native encoding and semantic quantization beyond llms. *arXiv preprint arXiv:2602.02338* (2026).
- [25] Siyi Lin, Chongming Gao, Jiawei Chen, Sheng Zhou, Binbin Hu, Yan Feng, Chun Chen, and Can Wang. 2025. How do recommendation models amplify popularity bias? An analysis from the spectral perspective. In *International ACM Conference on Web Search and Data Mining (WSDM)*. 659–668.
- [26] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2025. Generative recommender with end-to-end learnable item tokenization. In *International ACM Conference on Research and Development in Information Retrieval (SIGIR)*. 729–739.
- [27] Xinchun Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, et al. 2025. Qarm: Quantitative alignment multi-modal recommendation at kuaishou. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 5915–5922.
- [28] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. 2024. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations (ICLR)*, Vol. 2024. 51772–51783.
- [29] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.
- [30] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems (NeurIPS)* 36 (2023), 10299–10315.
- [31] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Conference on Uncertainty in Artificial Intelligence (UAI)*.
- [32] Yifei Shen, Yongji Wu, Yao Zhang, Caihua Shan, Jun Zhang, B Khaled Letaief, and Dongsheng Li. 2021. How powerful is graph convolution for recommendation?. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 1619–1629.
- [33] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 1441–1450.
- [34] Juntao Tan, Shuyuan Xu, Wenye Hua, Yingqiang Ge, Zelong Li, and Yongfeng Zhang. 2024. Idgenrec: Llm-recsys alignment with textual id learning. In *International ACM Conference on Research and Development in Information Retrieval (SIGIR)*. 355–364.
- [35] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *International ACM Conference on Web Search and Data Mining (WSDM)*. 565–573.
- [36] Matthew Thorpe, Tan Minh Nguyen, Hedi Xia, Thomas Strohmmer, Andrea Bertozzi, Stanley Osher, and Bao Wang. 2022. GRAND++: Graph neural diffusion with a source term. In *International Conference on Learning Representations (ICLR)*.
- [37] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. 2019. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports* 9, 1 (2019), 5233.
- [38] Aaron Van Den Oord, Oriol Vinyals, et al. 2017. Neural discrete representation learning. *Advances in Neural Information Processing Systems (NeurIPS)* 30 (2017).
- [39] Qiang Wan, Ze Yang, Dawei Yang, Ying Fan, Xin Yan, Siyang Liu, Yicong Liu, Chenwei Zhang, Wei Xu, Jiahao Qin, et al. 2026. R3-VAE: Reference vector-guided residual quantization VAE for generative recommendation. *arXiv preprint arXiv:2604.11440* (2026).
- [40] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable item tokenization for generative recommendation. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 2400–2409.
- [41] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *International ACM Conference on Research and Development in Information Retrieval (SIGIR)*. 165–174.
- [42] Shu Wu, Yuyuan Tang, Yanqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-based recommendation with graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 33. 346–353.
- [43] Cong Xu, Jun Wang, Jianyong Wang, and Wei Zhang. 2024. Graph-enhanced optimizers for structure-aware recommendation embedding evolution. *Advances in Neural Information Processing Systems (NeurIPS)* 37 (2024), 80828–80863.
- [44] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. In *International Conference on Machine Learning (ICML)*. PMLR, 58484–58509.

- [45] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.
- [46] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. 2025. OneRec-v2 technical report. *arXiv preprint arXiv:2508.20900* (2025).
- [47] Kaixiong Zhou, Xiao Huang, Daochen Zha, Rui Chen, Li Li, Soo-Hyun Choi, and Xia Hu. 2021. Dirichlet energy constrained learning for deep graph neural networks. *Advances in Neural Information Processing Systems (NeurIPS)* 34 (2021), 21834–21846.
- [48] Jieming Zhu, Mengqun Jin, Qijiong Liu, Zexuan Qiu, Zhenhua Dong, and Xiu Li. 2024. Cost: Contrastive quantization based semantic tokenization for generative recommendation. In *ACM Conference on Recommender Systems (RecSys)*. 969–974.
- [49] Yongxin Zhu, Bocheng Li, Yifei Xin, Zhihua Xia, and Linli Xu. 2025. Addressing representation collapse in vector quantized models with one linear layer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 22968–22977.

A Implementation Details

A.1 Experimental Details

We use the 5-core filtered Amazon Beauty, Toys, and Sports datasets and follow the leave-one-out protocol [19, 30]: the last and penultimate interactions of each user are reserved for test and validation, respectively. For each item, we concatenate its title, categories, and brand fields and encode the resulting text with sentence-t5-xl [30]. Dataset statistics are reported in Table 4.

Conventional recommenders are evaluated by ranking all items, and we report HR@5, HR@10, NDCG@5, and NDCG@10. Generative recommenders use constrained beam search with a beam size of 20. All results are averaged over five runs with seeds {0, 1, 2, 3, 4}. We tune each baseline independently on each dataset. For SDQ-VAE and RQ-VAE, Sinkhorn regularization [45] is applied only at the final quantization stage. SDQ-VAE and SDQ-KMeans use DOPRI5 for graph diffusion, while all generative recommenders use the same six-layer, four-head T5 decoder. The remaining SDQ-VAE and T5 settings are listed in Tables 5 and 6.

For the VAE-based quantizers in the TIGER framework, including PQ-VAE and all RQ-VAE variants, the tokenizer employs $K = 3$ codebooks, each containing $M = 256$ codewords, together with an additional verification code to prevent SID collisions. RQ-VAE variants are trained for 10,000 epochs by default, and the learning rate ([1e-4, 5e-4, 1e-3]), weight decay ([0, 1e-4, 1e-3, 1e-2]), Sinkhorn epsilon ([0.003, 0.03, 0.3]), and commitment weight ([0, 0.1, 0.25, 0.5, 1]) are tuned separately for each dataset.

A.2 Baselines

Conventional recommenders. MF [31] learns user and item embeddings from implicit feedback with the BPR pairwise ranking objective. **LightGCN** [12] propagates embeddings over the user-item interaction graph using simplified linear graph convolution. **Caser** [35] applies horizontal and vertical convolutional filters to capture sequential patterns. **GRU4Rec** [13] models interaction sequences with gated recurrent units. **SASRec** [19] uses causal self-attention to model sequential dependencies. **BERT4Rec** [33] adopts bidirectional Transformer encoding with masked-item prediction. **Constraint-guided generative recommenders.** **ETEGRec** [26] jointly optimizes an item tokenizer and an autoregressive recommender through sequence-item and preference-semantic alignment objectives. **LETTER** [40] regularizes RQ-VAE tokenization

Table 4: Dataset statistics.

Dataset	#Users	#Items	#Interactions	Avg. Len.
Beauty	22,363	12,101	198,502	8.88
Toys	19,412	11,924	167,597	8.63
Sports	35,598	18,357	296,337	8.32

Table 5: Hyperparameter settings for SDQ-VAE.

Hyperparameter	Beauty	Toys	Sports
Codewords per codebook (M)	256	256	256
Codebooks (K)	3	3	3
Codebook dimension (d)	128	128	128
Encoder dimensions	512, 128	512, 128	512, 128
Sinkhorn epsilon	0.003	0.003	0.003
Learning rate	5×10^{-4}	5×10^{-4}	1×10^{-4}
Weight decay	0	0	0
Dropout rate	0	0	0
Batch size	256	256	256
Epochs	100	100	100

with semantic, collaborative, and diversity objectives, and further introduces a ranking-guided generation loss. **DIGER** [10] makes SIDs differentiable to align indexing and recommendation objectives, using Gumbel exploration with uncertainty decay to mitigate codebook collapse. **R3-VAE** [39] uses reference vectors and a dot-product rating mechanism to stabilize RQ-VAE, together with semantic-cohesion and preference-discrimination regularization. **TIGER-framework quantization variants.** **PQ-KMeans** [17] partitions an item representation into subspaces and independently quantizes each subvector with KMeans. **RQ-KMeans** [27] applies KMeans successively to residuals to form multi-token item identifiers. **PQ-VAE** [38] combines an encoder-decoder architecture with product quantization over latent subspaces. **RQ-VAE (STE)** [30] is the standard residual VAE tokenizer trained with straight-through gradients for hard nearest-neighbor assignments. **RQ-VAE (Gumbel)** [16] replaces hard assignment training with Gumbel-Softmax reparameterization. **RQ-VAE (Rotation)** [9] modifies the backward gradient through quantization with the rotation trick. **RQ-VAE (SimVQ)** [49] couples code-vector updates through a shared linear transformation to alleviate representation collapse.

A.3 Graph Construction

As downstream generative recommendation is inherently a sequential recommendation task, collaborative information from user-item interactions is particularly important. This motivates previous studies to develop various training objectives for injecting collaborative signals during item tokenization. In contrast, we show that SDQ can serve as an implicit structure prior injector through graph diffusion. All experiments use the co-occurrence graph constructed by Algorithm 1. We extract adjacent item pairs from the training sequences, retain both directed edges with unit weights [42, 43], and add a self-loop to every item.

Table 6: Shared hyperparameter settings for T5.

Hyperparameter	Value
Maximum sequence length	20
Embedding dimension	128
Attention dimension	64
Intermediate dimension	256
Transformer layers	6
Attention heads	4
Dropout	0.1
Optimizer	AdamW
Learning rate	5×10^{-4}
Weight decay	10^{-3}
Batch size	512
Epochs	200

Algorithm 1: Co-occurrence graph construction.**Input:** Item set $\mathcal{V}$ and training sequences $\mathcal{S}$ **Output:** Item graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ $\mathcal{E} \leftarrow \emptyset;$ **foreach** (i, j) *in the adjacent pairs of every* $s \in \mathcal{S}$ **do** $\mathcal{E} \leftarrow \mathcal{E} \cup \{(i, j), (j, i)\};$ $\mathcal{E} \leftarrow \mathcal{E} \cup \{(i, i) : i \in \mathcal{V}\};$ **return** $\mathcal{G} = (\mathcal{V}, \mathcal{E});$ **Algorithm 2: Batch-induced training subgraph construction.****Input:** Global item graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and mini-batch sequences $\mathcal{B}$ **Output:** Normalized local adjacency $\tilde{\mathbf{A}}_{\mathcal{B}}$ $\mathcal{V}_{\mathcal{B}} \leftarrow \bigcup_{s \in \mathcal{B}} \{i : i \in s\};$ $\mathcal{E}_{\mathcal{B}} \leftarrow \{(i, j) \in \mathcal{E} : i, j \in \mathcal{V}_{\mathcal{B}}\};$ $\mathbf{A}_{\mathcal{B}} \leftarrow \text{Adjacency}(\mathcal{V}_{\mathcal{B}}, \mathcal{E}_{\mathcal{B}});$ $\mathbf{D}_{\mathcal{B}} \leftarrow \text{diag}(\mathbf{A}_{\mathcal{B}} \mathbf{1});$ $\tilde{\mathbf{A}}_{\mathcal{B}} \leftarrow \mathbf{D}_{\mathcal{B}}^{-1/2} \mathbf{A}_{\mathcal{B}} \mathbf{D}_{\mathcal{B}}^{-1/2};$ **return** $\tilde{\mathbf{A}}_{\mathcal{B}};$ **A.4 Training Subgraph Construction**

For SDQ-VAE training, diffusion is performed on the normalized induced subgraph of the items appearing in the current mini-batch rather than on the full graph. Algorithm 2 describes this procedure. SDQ-KMeans and SID generation use the full item graph; in particular, all reported SDQ-VAE results use full-graph inference.

B Discrete Forward and Backward Diffusion

For the linear system $\dot{\mathbf{X}}(t) = -\tilde{\mathbf{L}}\mathbf{X}(t)$, forward discretization from t_0 to t_1 with step size Δt is equivalent to evaluating a matrix polynomial at $-\Delta t \tilde{\mathbf{L}}$. In contrast, backward integration uses a step size

of $-\Delta t$ and therefore evaluates the same polynomial at $\Delta t \tilde{\mathbf{L}}$. Specifically, the corresponding polynomials for the Euler, RK4, and DOPRI5 schemes are given by

$$p_{\text{Euler}}(\mathbf{Z}) = \mathbf{I} + \mathbf{Z}, \quad (8)$$

$$p_{\text{RK4}}(\mathbf{Z}) = \mathbf{I} + \mathbf{Z} + \frac{\mathbf{Z}^2}{2} + \frac{\mathbf{Z}^3}{6} + \frac{\mathbf{Z}^4}{24}, \quad (9)$$

$$p_{\text{DOPRI5}}(\mathbf{Z}) = \mathbf{I} + \mathbf{Z} + \frac{\mathbf{Z}^2}{2} + \frac{\mathbf{Z}^3}{6} + \frac{\mathbf{Z}^4}{24} + \frac{\mathbf{Z}^5}{120} + \frac{\mathbf{Z}^6}{600}. \quad (10)$$

We then obtain the following discretization formulas:

- **Exact flow.** The continuous forward and backward solutions are

$$\Phi_{t_0 \rightarrow t_1}(\mathbf{X}) = \exp(-(t_1 - t_0)\tilde{\mathbf{L}})\mathbf{X},$$

$$\Phi_{t_1 \rightarrow t_0}(\mathbf{X}) = \exp((t_1 - t_0)\tilde{\mathbf{L}})\mathbf{X} = \mathbf{B}_{t_1 \rightarrow t_0}^{\text{Exact}} \mathbf{X}.$$

- **Euler discretization.** For N uniform steps with $\Delta t = (t_1 - t_0)/N$,

$$\hat{\Phi}_{t_0 \rightarrow t_1}^{\text{Euler}}(\mathbf{X}) = p_{\text{Euler}}(-\Delta t \tilde{\mathbf{L}})^N \mathbf{X} = (\mathbf{I} - \Delta t \tilde{\mathbf{L}})^N \mathbf{X},$$

$$\hat{\Phi}_{t_1 \rightarrow t_0}^{\text{Euler}}(\mathbf{X}) = p_{\text{Euler}}(\Delta t \tilde{\mathbf{L}})^N \mathbf{X} = (\mathbf{I} + \Delta t \tilde{\mathbf{L}})^N \mathbf{X} = \mathbf{B}_{t_1 \rightarrow t_0}^{\text{Euler}} \mathbf{X}.$$

- **RK4 discretization.** Under the same uniform partition,

$$\hat{\Phi}_{t_0 \rightarrow t_1}^{\text{RK4}}(\mathbf{X}) = p_{\text{RK4}}(-\Delta t \tilde{\mathbf{L}})^N \mathbf{X},$$

$$\hat{\Phi}_{t_1 \rightarrow t_0}^{\text{RK4}}(\mathbf{X}) = p_{\text{RK4}}(\Delta t \tilde{\mathbf{L}})^N \mathbf{X} = \mathbf{B}_{t_1 \rightarrow t_0}^{\text{RK4}} \mathbf{X}.$$

- **DOPRI5 discretization.** The adaptive forward and backward integrations may accept different step sequences. Let Δt_s^f and Δt_s^b denote their respective step sizes, each summing to $t_1 - t_0$ over the corresponding accepted steps. We have,

$$\hat{\Phi}_{t_0 \rightarrow t_1}^{\text{DOPRI5}}(\mathbf{X}) = \prod_{s=1}^{N^f} p_{\text{DOPRI5}}(-\Delta t_s^f \tilde{\mathbf{L}}) \mathbf{X},$$

$$\hat{\Phi}_{t_1 \rightarrow t_0}^{\text{DOPRI5}}(\mathbf{X}) = \prod_{s=1}^{N^b} p_{\text{DOPRI5}}(\Delta t_s^b \tilde{\mathbf{L}}) \mathbf{X} = \mathbf{B}_{t_1 \rightarrow t_0}^{\text{DOPRI5}} \mathbf{X}.$$

For notational simplicity, we explicitly denote the backward matrix polynomial by $\mathbf{B}$ in the following.

C Proofs of Codebook Analysis

PROOF OF PROPOSITION 3.1. Under fixed assignments and solver steps, $\mathbf{P}_k$ and $\mathbf{B}_k$ are constant with respect to $\mathbf{C}_k$. The stop-gradient operations in the first and third terms of $\mathcal{L}_{\text{SDQ}}$ block their codebook gradients. Using the existing identity $\mathbf{R}_{\ell+1} = \mathbf{X} - \tilde{\mathbf{X}}_{\ell}$, the codebook-dependent term of the unified objective is therefore $\sum_{\ell=1}^K \|\mathbf{R}_{\ell+1}\|_F^2$.

For $\ell \geq k$, the contribution recovered from $\mathbf{C}_k$ is $\mathbf{B}_k \mathbf{P}_k \mathbf{C}_k$, and hence

$$d\mathbf{R}_{\ell+1} = -\mathbf{B}_k \mathbf{P}_k d\mathbf{C}_k. \quad (11)$$

For $\ell < k$, this differential is zero. It follows that

$$d\mathcal{L}_{\text{SDQ}} = -2 \sum_{\ell=k}^K \text{Tr}[(\mathbf{P}_k^T \mathbf{B}_k^T \mathbf{R}_{\ell+1})^T d\mathbf{C}_k], \quad (12)$$

and hence

$$\nabla_{\mathbf{C}_k} \mathcal{L}_{\text{SDQ}} = -2 \sum_{\ell=k}^K \mathbf{P}_k^T \mathbf{B}_k^T \mathbf{R}_{\ell+1}. \quad (13)$$

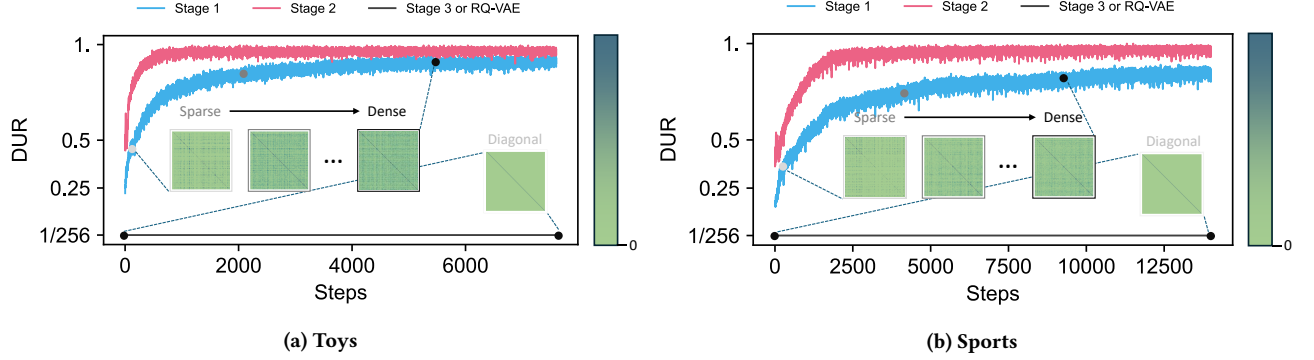


Figure 9: Codebook data utilization ratio (DUR) on Toys and Sports. The first two SDQ codebooks progressively receive supervision from a broad set of items and exhibit increasingly dense $\mathbf{P}^\top \mathbf{B}^\top \mathbf{B} \mathbf{P}$, whereas the final stage and RQ-VAE retain the direct-assignment, diagonal structure.

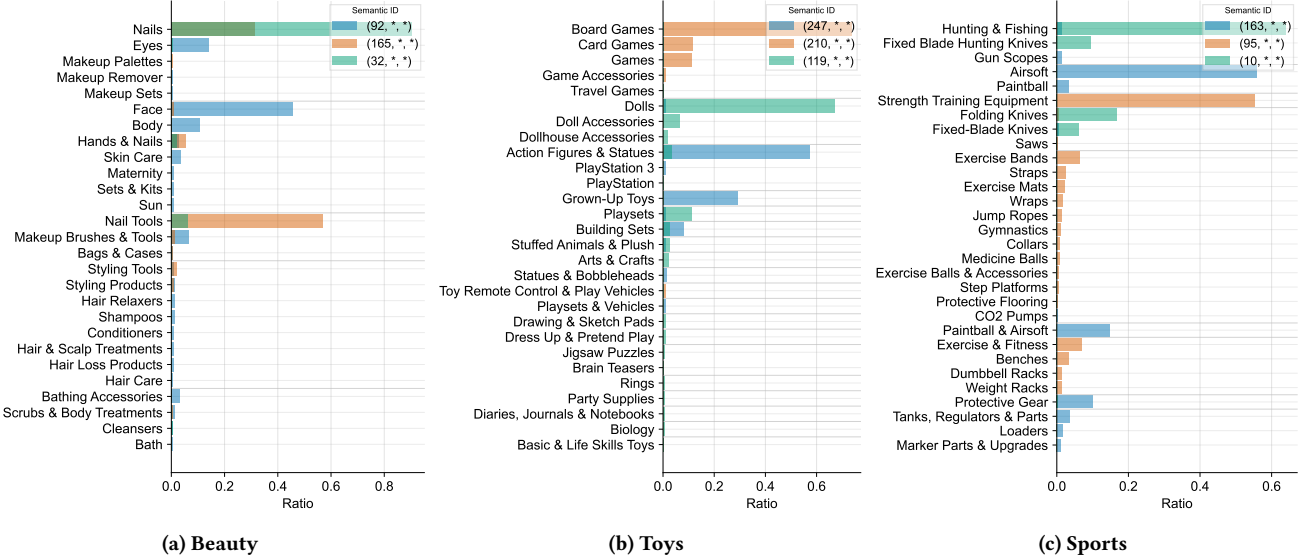


Figure 10: Category distributions of items assigned to three representative first-position SID prefixes ($s_1, *, *$) on Beauty, Toys, and Sports. Each color denotes one SID prefix, and the bar length is its ratio within the corresponding product category.

Moreover,

$$\mathbf{R}_{\ell+1} = \mathbf{X} - \sum_{j=1}^{\ell} \mathbf{B}_j \mathbf{P}_j \mathbf{C}_j. \quad (14)$$

Substituting this expression into (13) yields

$$\nabla_{\mathbf{C}_k} \mathcal{L}_{\text{SDQ}} = 2 \sum_{\ell=k}^K \sum_{j=1}^{\ell} \mathbf{P}_k^\top \mathbf{B}_k^\top \mathbf{B}_j \mathbf{P}_j \mathbf{C}_j - 2(K-k+1) \mathbf{P}_k^\top \mathbf{B}_k^\top \mathbf{X}. \quad (15)$$

For a fixed j , the corresponding codebook term appears for every $\ell \in \{\max\{k, j\}, \dots, K\}$, namely $K - \max\{k, j\} + 1$ times. Collecting these terms proves (6). $\square$

PROOF OF COROLLARY 3.3. When $\tilde{\mathbf{L}} = \mathbf{0}$, all backward operators in Appendix B reduce to $\mathbf{I}$. Substituting $\mathbf{B}_j = \mathbf{I}$ for every j into (6)

gives

$$\nabla_{\mathbf{C}_k} \mathcal{L}_{\text{SDQ}} = 2 \sum_{j=1}^K (K - \max\{k, j\} + 1) \mathbf{P}_k^\top \mathbf{P}_j \mathbf{C}_j - 2(K-k+1) \mathbf{P}_k^\top \mathbf{X}.$$

Since each row of the one-hot matrix $\mathbf{P}_k$ contains exactly one nonzero entry, its columns have disjoint supports, $\mathbf{P}_k^\top \mathbf{P}_k$ is diagonal, and $\|\mathbf{P}_k^\top\|_0 = |\mathcal{V}|$. Setting $\mathbf{B}_k = \mathbf{I}$ in Definition 3.2 therefore gives $\text{DUR}(\mathbf{C}_k) = 1/M$. $\square$

D Additional Empirical Analysis

D.1 Data Utilization Across Datasets

Figure 9 confirms that the DUR behavior observed on Beauty is consistent across Toys and Sports. The first two SDQ codebooks rapidly reach high DUR, while the associated coefficient matrices

Table 7: Wall-clock training time (s) for quantization and downstream generative-model training on a single NVIDIA H20 GPU. The downstream T5 training time is measured with a fixed budget of 200 epochs.

	Beauty	Toys	Sports
PQ-KMeans	3.1	2.9	3.5
RQ-KMeans	5.8	5.4	8.5
SDQ-KMeans	9.2	9.2	12.9
PQ-VAE	2017.9	2012.3	3042.0
STE	1960.6	1952.6	2952.5
Gumbel	2020.4	2020.5	3050.1
Rotation	2054.7	2042.6	3028.2
SimVQ	1985.0	1954.0	2963.6
SDQ-VAE	1333.3	1110.5	2172.9
Generative Training (T5)	4432.7	3670.9	6629.6

become progressively denser during training. By contrast, the final quantization stage has zero diffusion time and therefore coincides with the direct assignments of RQ-VAE, yielding the diagonal structure and DUR of $1/M$. These results further support that backward diffusion broadens supervision for the earlier codebooks across datasets.

D.2 Effect of the Quantization Loss

We investigate the effect of the quantization objective. When training SDQ-VAE with the conventional RQ-VAE objective [18, 38], we replace $\mathcal{L}_{\text{SDQ}}$ with the stage-wise loss

$$\begin{aligned} \mathcal{L}_{\text{RQ}} = & \|\hat{\mathbf{H}} - \mathbf{H}\|_F^2 \\ & + \sum_{k=1}^K \left[\left\| \mathbf{P}_k \mathbf{C}_k - \text{sg}(\hat{\Phi}_{t_0 \rightarrow t_1}(\mathbf{R}_k)) \right\|_F^2 \right. \\ & \left. + \beta \left\| \hat{\Phi}_{t_0 \rightarrow t_1}(\mathbf{R}_k) - \text{sg}(\mathbf{P}_k \mathbf{C}_k) \right\|_F^2 \right]. \end{aligned}$$

The RQ objective matches the k -th codebook only to its own forward-diffused residual. Consequently, under fixed assignments and solver steps, its effective codebook supervision operator is $\mathbf{P}_k^\top$, and the corresponding coefficient $\mathbf{P}_k^\top \mathbf{P}_k$ is diagonal because $\mathbf{P}_k$ is one-hot. Each code vector therefore receives direct supervision only from its assigned items, yielding the direct-assignment DUR of $1/M$ and no joint learning. In contrast, $\mathcal{L}_{\text{SDQ}}$ supervises every partial recovered estimate $\hat{\mathbf{X}}_k$ against the original representation $\mathbf{X}$. Its codebook gradient consequently contains $\mathbf{P}_k^\top \mathbf{B}_k^\top$ and the non-diagonal coefficient matrix $\mathbf{P}^\top \mathbf{B}^\top \mathbf{B} \mathbf{P}$ (Proposition 3.1). Backward recovery therefore propagates the supervision of structurally related items to earlier code vectors, increasing their effective data utilization while jointly optimizing the codebooks.

D.3 Wall-Clock Efficiency

Table 7 shows that SDQ-KMeans incurs a small additional cost over RQ-KMeans because of numerical diffusion, while remaining inexpensive in absolute time. For VAE quantization, SDQ-VAE is consistently faster than all RQ-VAE variants, as its markedly shorter

convergence horizon outweighs part of the per-iteration discretization cost. Nevertheless, this improvement is not proportional to the reduction in training epochs, and downstream T5 training remains the dominant cost under the fixed 200-epoch budget.

D.4 Semantic Coherence of SIDs

Figure 10 visualizes three representative first-position SIDs for each dataset. The three prefixes specialize in distinct yet semantically coherent category groups, including face and nail products on Beauty, dolls and action figures on Toys, and shooting or exercise equipment on Sports. This category-level concentration provides qualitative evidence that the early SID position captures broad semantic patterns, while later positions can be used to refine item-level distinctions.

D.5 Limitations

SDQ can be further improved in several aspects. First, its current diffusion schedule follows a manually specified stage-wise rule. Although this design is simple and robust across our experiments, it may be suboptimal for datasets with substantially different graph densities, spectral properties, or desired levels of hierarchy granularity. Learning data-adaptive or stage-adaptive diffusion times could enable more flexible hierarchy formation [5].

Second, SDQ relies on accurate forward-backward numerical integration. Higher-order or adaptive solvers can reduce discretization errors, but they also incur additional propagation costs. Although SDQ remains highly efficient during training, its practical scalability could be further enhanced by developing more suitable solvers that better balance numerical accuracy and computational efficiency.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009